

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

ЛАБОРАТОРНАЯ РАБОТА №1

По дисциплине Промышленная маршрутизация и управление трафиком

Тема работы Установка ContainerLab и развертывание тестовой сети связи

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа K3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Аминов Н.С.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 ХОД РАБОТЫ.....	4
1.1 Подготовка.....	4
1.2 Создание сети связи	6
1.3 Проверка пингов и локальной связности	11
ЗАКЛЮЧЕНИЕ	14

ВВЕДЕНИЕ

В отчёте представлен ход выполнения и результат работы.

Цель - ознакомиться с инструментом ContainerLab и методами работы с ним, изучить работу VLAN, IP адресации и т.д.

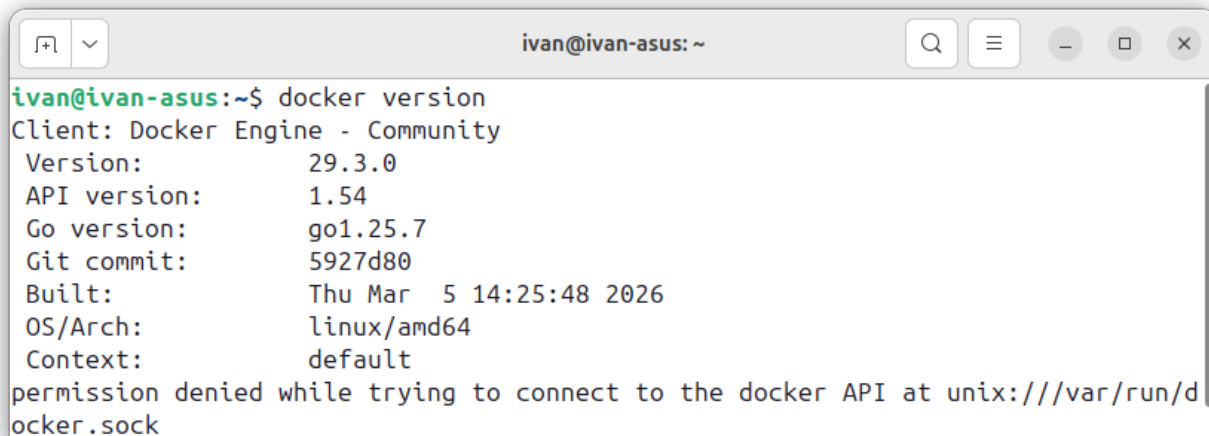
Задачи:

- установить docker, make, containerlab;
- собрать контейнер с RouterOS;
- собрать трёхуровневую сеть связи;
- настроить маршрутизатор и коммутаторы.

1 ХОД РАБОТЫ

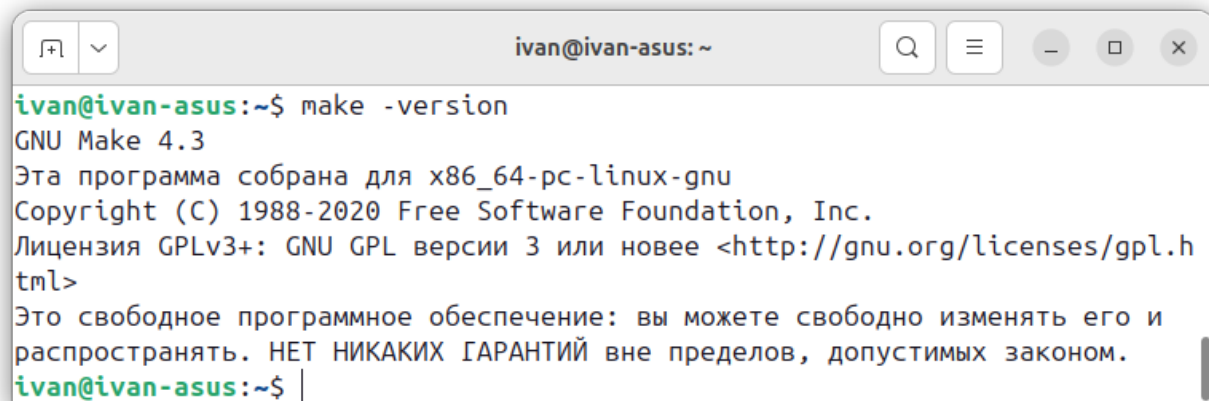
1.1 Подготовка

Лабораторная работа выполнялась на linux на дистрибутиве ubuntu. Программы docker и make уже были установлены (рисунки 1 и 2).



```
ivan@ivan-asus:~$ docker version
Client: Docker Engine - Community
Version:      29.3.0
API version:  1.54
Go version:   go1.25.7
Git commit:   5927d80
Built:        Thu Mar  5 14:25:48 2026
OS/Arch:      linux/amd64
Context:      default
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
```

Рисунок 1 — Проверка установки docker



```
ivan@ivan-asus:~$ make -version
GNU Make 4.3
Эта программа собрана для x86_64-pc-linux-gnu
Copyright (C) 1988-2020 Free Software Foundation, Inc.
Лицензия GPLv3+: GNU GPL версии 3 или новее <http://gnu.org/licenses/gpl.h
tml>
Это свободное программное обеспечение: вы можете свободно изменять его и
распространять. НЕТ НИКАКИХ ГАРАНТИЙ вне пределов, допустимых законом.
ivan@ivan-asus:~$ |
```

Рисунок 2 — Проверка установки make

Для сборки контейнера с виртуальной машиной Mikrotic на RouterOS был клонирован репозиторий с проектом vnetlab (рисунок 3), добавлен виртуальный жесткий диск (рисунок 4) и собран контейнер (рисунок 5).

```
ivan@ivan-asus: ~/pmut
ivan@ivan-asus:~/pmut$ git clone https://github.com/srl-labs/vrnetlab.git
Клонирование в «vrnetlab»...
remote: Enumerating objects: 6272, done.
remote: Counting objects: 100% (1003/1003), done.
remote: Compressing objects: 100% (141/141), done.
remote: Total 6272 (delta 908), reused 868 (delta 860), pack-reused 5269 (from 2)
Получение объектов: 100% (6272/6272), 4.03 МиБ | 10.45 МиБ/с, готово.
Определение изменений: 100% (3743/3743), готово.
```

Рисунок 3 — Клонирование vrnetlab

```
ivan@ivan-asus: ~/pmut/vrnetlab/mikrotik/routeros
ivan@ivan-asus:~/pmut$ cd ./vrnetlab/mikrotik/routeros/
ivan@ivan-asus:~/pmut/vrnetlab/mikrotik/routeros$ curl -O https://download.mikro
tik.com/routeros/6.47.9/chr-6.47.9.vmdk
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 34.0M  100 34.0M    0     0  28.1M      0  0:00:01  0:00:01 --:--:-- 28.1M
ivan@ivan-asus:~/pmut/vrnetlab/mikrotik/routeros$ ls
chr-6.47.9.vmdk  docker  Makefile  README.md
```

Рисунок 4 — Добавление виртуального жесткого диска

```
ivan@ivan-asus: ~/pmut/vrnetlab/mikrotik/routeros
ivan@ivan-asus:~$ cd ./pmut/vrnetlab/mikrotik/routeros/
ivan@ivan-asus:~/pmut/vrnetlab/mikrotik/routeros$ make docker-image
Makefile:28: предупреждение: переопределение способа для цели «docker-build-extra»
../../makefile.include:56: предупреждение: старый способ для цели «docker-build-extra» игнорируются
Makefile:36: предупреждение: переопределение способа для цели «docker-push-image-extra»
../../makefile.include:66: предупреждение: старый способ для цели «docker-push-image-extra» игнорируются
for IMAGE in chr-6.47.9.vmdk; do \
    echo "Making $IMAGE"; \
    make IMAGE=$IMAGE docker-build; \
    make IMAGE=$IMAGE docker-clean-build; \
done
```

Рисунок 5 — Сборка контейнера

Наконец, был установлен containerlab (рисунок 6), а также добавлен `alias clab='containerlab'`.

[illegible]

Рисунок 6 — Установка containerlab

1.2 Создание сети связи

На рисунке 7 представлена конфигурация сети. Создан маршрутизатор, три коммутатора (на базе ROS) и два компьютера (на базе alpine linux), сеть управления (mgmt) и связи между устройствами (links).

```
1 name: lab1
2 mgmt:
3   network: mgmt
4   ipv4-subnet: 172.20.20.0/24
5 topology:
6   nodes:
7     R01.TEST:
8       kind: vr-ros
9       image: vrnetlab/mikrotik_routeros:6.47.9
10      mgmt-ipv4: 172.20.20.10
11     SW01.L3.01.TEST:
12       kind: vr-ros
13       image: vrnetlab/mikrotik_routeros:6.47.9
14       mgmt-ipv4: 172.20.20.11
15     SW02.L3.01.TEST:
16       kind: vr-ros
17       image: vrnetlab/mikrotik_routeros:6.47.9
18       mgmt-ipv4: 172.20.20.21
19     SW02.L3.02.TEST:
20       kind: vr-ros
21       image: vrnetlab/mikrotik_routeros:6.47.9
22       mgmt-ipv4: 172.20.20.22
23     PC1:
24       kind: linux
25       image: alpine:latest
26       mgmt-ipv4: 172.20.20.31
27       cmd: sleep infinity
28     PC2:
29       kind: linux
30       image: alpine:latest
31       mgmt-ipv4: 172.20.20.32
32       cmd: sleep infinity
33 links:
34   - endpoints: ["R01.TEST:eth1", "SW01.L3.01.TEST:eth1"]
35   - endpoints: ["SW01.L3.01.TEST:eth2", "SW02.L3.01.TEST:eth1"]
36   - endpoints: ["SW01.L3.01.TEST:eth3", "SW02.L3.02.TEST:eth1"]
37   - endpoints: ["SW02.L3.01.TEST:eth2", "PC1:eth1"]
38   - endpoints: ["SW02.L3.02.TEST:eth2", "PC2:eth1"]
```

Рисунок 7 — Конфигурация сети в containerlab
После написания конфигурации сеть была запущена (рисунок 8).

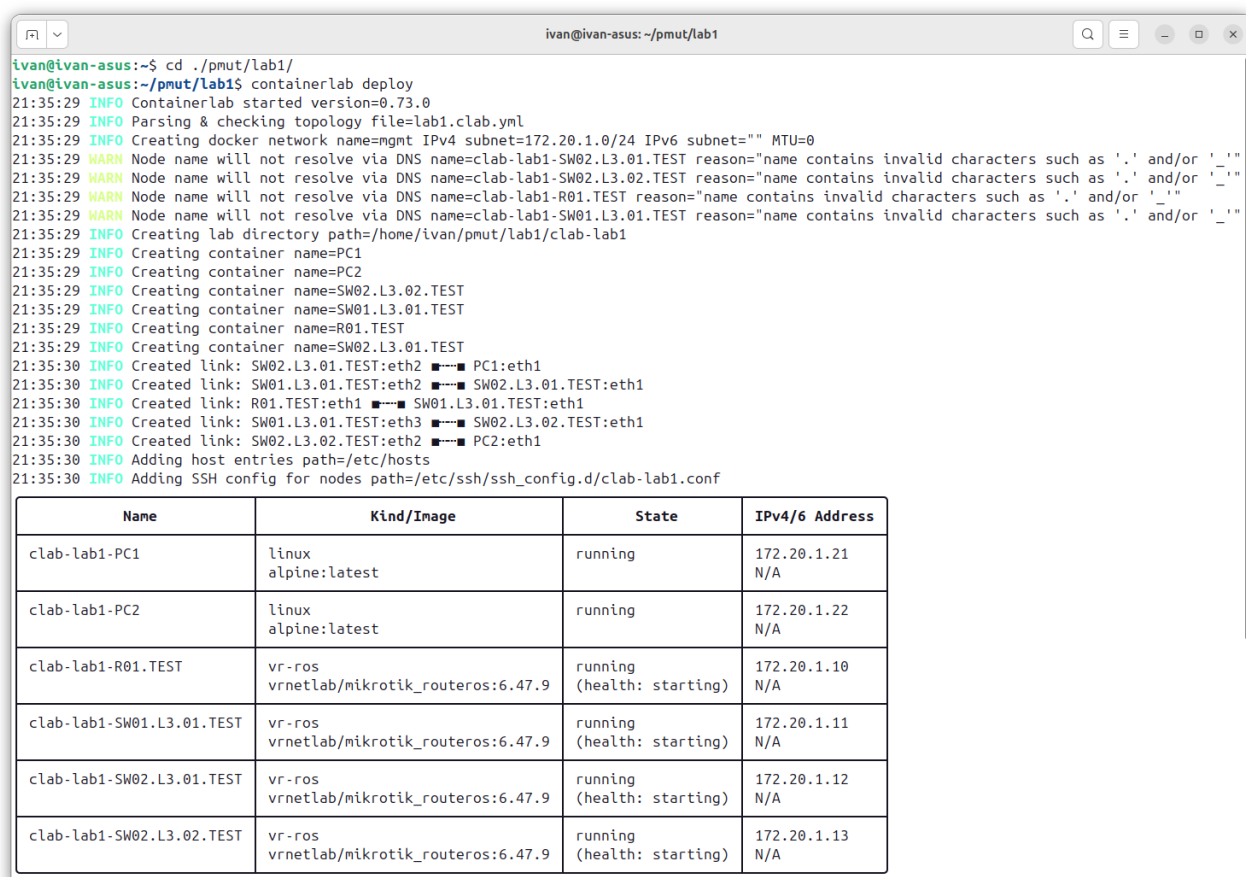


Рисунок 8 — Развёртывание сети в containerlab

Для установки начальных конфигураций сначала был использован startup-config, однако он не обновлял скрипты при пересоздании сети (clab redeploy). Затем был опробован способ с ожиданием загрузки, но он замораживал процесс пересоздания сети. По итогу, в директории с конфигурацией были обнаружены файлы config.auto.rsc, при запуске все команды по настройке из них по очереди выполнялись на нужных устройствах. Именно там были прописана конфигурация маршрутизатора и коммутаторов.

Конфигурация маршрутизатора представлена на рисунке 9. Сначала, задаётся имя, логин и пароль, затем создаются VLAN 10 и 20, на них назначаются адреса в соответствующих подсетях. Для каждого настраивается dhcp сервер в своей подсети.

```
1 /system identity set name=R01.TEST
2 /user set [find name=admin] password=hjenth
3
4 /interface vlan
5 add name=vlan10 vlan-id=10 interface=ether2
6 add name=vlan20 vlan-id=20 interface=ether2
7
8 /ip address
9 add address=192.168.10.1/24 interface=vlan10
10 add address=192.168.20.1/24 interface=vlan20
11
12 /ip pool add name=pool10 ranges=192.168.10.100-192.168.10.253
13 /ip dhcp-server add name=dhcp10 interface=vlan10 address-pool=pool10
14 /ip dhcp-server network add address=192.168.10.0/24 gateway=192.168.10.1
15 /ip dhcp-server enable dhcp10
16
17 /ip pool add name=pool20 ranges=192.168.20.100-192.168.20.253
18 /ip dhcp-server add name=dhcp20 interface=vlan20 address-pool=pool20
19 /ip dhcp-server network add address=192.168.20.0/24 gateway=192.168.20.1
20 /ip dhcp-server enable dhcp20
```

Рисунок 9 — Настройки маршрутизатора R01.TEST

Центральный распределительный коммутатор настраивается как bridge с trunk (tagged) портами (рисунок 10).

```
1 /system identity set name=SW01.L3.01.TEST
2 /user set [find name=admin] password=wtynh
3
4 /interface bridge add name=bridge1 vlan-filtering=yes
5 /interface bridge port
6 add bridge=bridge1 interface=ether2
7 add bridge=bridge1 interface=ether3
8 add bridge=bridge1 interface=ether4
9 /interface bridge vlan
10 add bridge=bridge1 vlan-ids=10 tagged=ether2,ether3
11 add bridge=bridge1 vlan-ids=20 tagged=ether2,ether4
```

Рисунок 10 — Настройки центрального коммутатора SW01.L3.01.TEST

На рисунках 11 и 12 представлены настройки для двух коммутаторов доступа. Левый предоставляет access (untagged) соединение для PC1, правый – для PC2.

```
1 /system identity set name=SW02.L3.01.TEST
2 /user set [find name=admin] password=kt dsq
3
4 /interface bridge add name=bridge1 vlan-filtering=yes
5 /interface bridge port
6 add bridge=bridge1 interface=ether2
7 add bridge=bridge1 interface=ether3 pvid=10
8 /interface bridge vlan
9 add bridge=bridge1 vlan-ids=10 tagged=ether2 untagged=ether3
10 add bridge=bridge1 vlan-ids=20 tagged=ether2
```

Рисунок 11 — Настройки левого коммутатора SW02.L3.01.TEST

```
1 /system identity set name=SW02.L3.02.TEST
2 /user set [find name=admin] password=ghf dsq
3
4 /interface bridge add name=bridge1 vlan-filtering=yes
5 /interface bridge port
6 add bridge=bridge1 interface=ether2
7 add bridge=bridge1 interface=ether3 pvid=20
8 /interface bridge vlan
9 add bridge=bridge1 vlan-ids=10 tagged=ether2
10 add bridge=bridge1 vlan-ids=20 tagged=ether2 untagged=ether3
```

Рисунок 12 — Настройки правого коммутатора SW02.L3.02.TEST

Для PC настройки следующие: удаляем mgmt адрес и шлюза по умолчанию, запрашиваем ip динамически (рисунок 13)

```
1 $ docker exec -it clab-lab1-PC1 sh
2 sh -c "ip route del default via 172.20.20.1 dev eth0 2>/dev/null; ip route add
   ↪ 172.20.20.0/24 dev eth0 2>/dev/null"
3 udhcpc -i eth1
```

Рисунок 13 — Конфигурация PC

По настроенной сети была нарисована схема связи (рисунок 14).

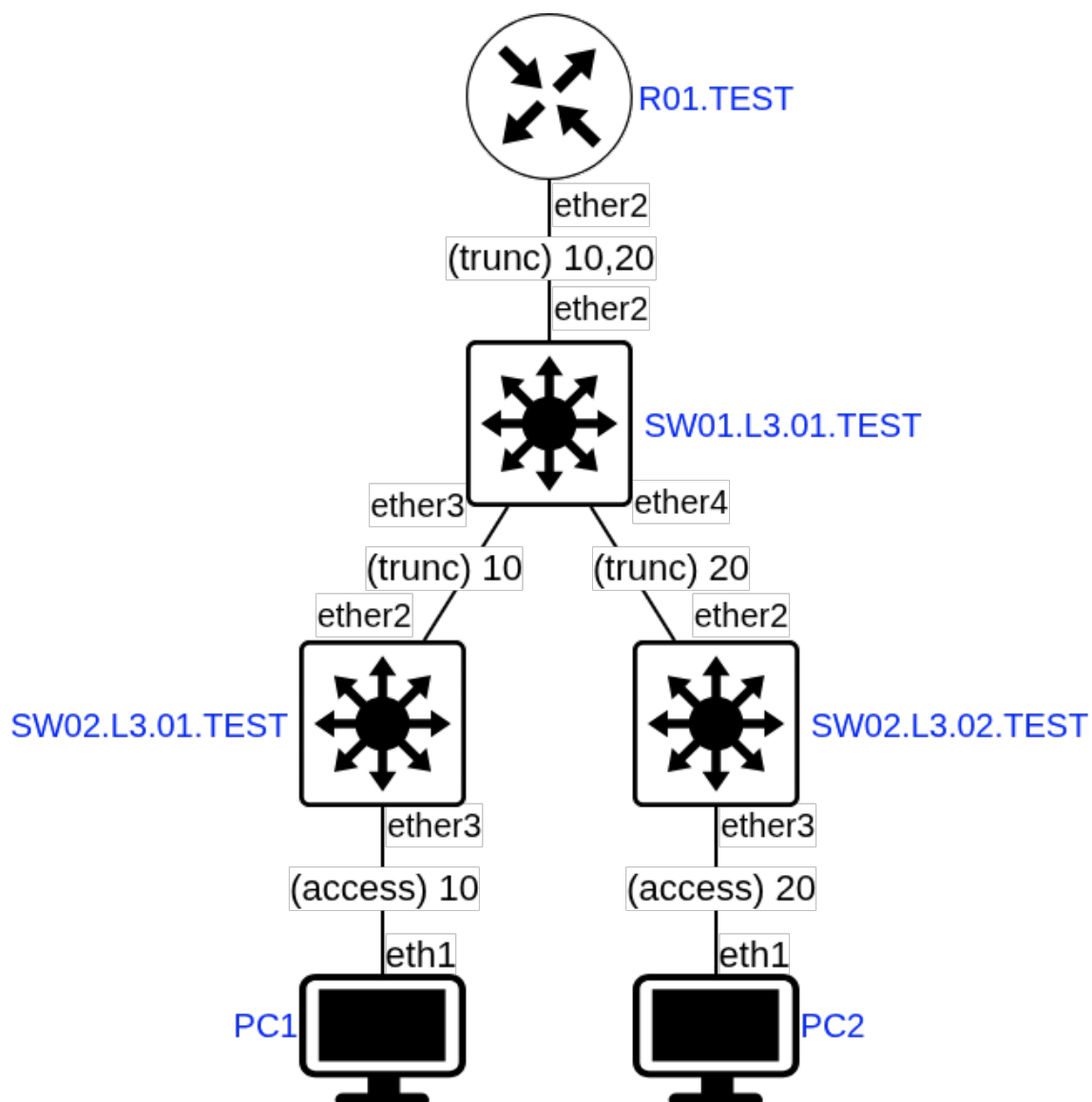


Рисунок 14 — Схема связи

1.3 Проверка пингов и локальной связности

Для проверки на компьютерах был проверен выданный динамически адрес и проведён пинг (рисунки 15 и 16). На маршрутизаторе проверены подсети, dhcp и ping (рисунок 17). На коммутаторах проверены настройки bridge и порты, на рисунке 18 представлены результаты для центрального, левый и правый аналогично.

```
ivan@ivan-asus: ~/pmut/lab1
ivan@ivan-asus:~/pmut/Lab1$ docker exec -it clab-lab1-PC1 sh
/ # ip addr show eth1
46: eth1@if47: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 9500 qdisc noqueue state UP
    link/ether aa:c1:ab:f3:0d:ef brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.253/24 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::a8c1:abff:fe3:def/64 scope link
        valid_lft forever preferred_lft forever
/ # ip route show
default via 192.168.10.1 dev eth1 metric 246
172.20.20.0/24 dev eth0 scope link src 172.20.20.31
192.168.10.0/24 dev eth1 scope link src 192.168.10.253
/ # ping -c 4 192.168.10.1
PING 192.168.10.1 (192.168.10.1): 56 data bytes
64 bytes from 192.168.10.1: seq=0 ttl=64 time=0.693 ms
64 bytes from 192.168.10.1: seq=1 ttl=64 time=1.361 ms
64 bytes from 192.168.10.1: seq=2 ttl=64 time=1.510 ms
64 bytes from 192.168.10.1: seq=3 ttl=64 time=1.465 ms

--- 192.168.10.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.693/1.257/1.510 ms
/ # ping -c 4 192.168.20.253
PING 192.168.20.253 (192.168.20.253): 56 data bytes
64 bytes from 192.168.20.253: seq=0 ttl=63 time=0.939 ms
64 bytes from 192.168.20.253: seq=1 ttl=63 time=1.467 ms
64 bytes from 192.168.20.253: seq=2 ttl=63 time=2.156 ms
64 bytes from 192.168.20.253: seq=3 ttl=63 time=2.023 ms

--- 192.168.20.253 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.939/1.646/2.156 ms
/ #
```

Рисунок 15 — Проверка PC1

```
ivan@ivan-asus: ~/pmut/lab1
ivan@ivan-asus:~/pmut/Lab1$ docker exec -it clab-lab1-PC2 sh
/ # ip addr show eth1
44: eth1@if45: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 9500 qdisc noqueue state UP
    link/ether aa:c1:ab:30:0c:02 brd ff:ff:ff:ff:ff:ff
    inet 192.168.20.253/24 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::a8c1:abff:fe30:c02/64 scope link
        valid_lft forever preferred_lft forever
/ # ip route show
default via 192.168.20.1 dev eth1 metric 244
172.20.20.0/24 dev eth0 scope link src 172.20.20.32
192.168.20.0/24 dev eth1 scope link src 192.168.20.253
/ # ping -c 4 192.168.20.1
PING 192.168.20.1 (192.168.20.1): 56 data bytes
64 bytes from 192.168.20.1: seq=0 ttl=64 time=0.666 ms
64 bytes from 192.168.20.1: seq=1 ttl=64 time=0.670 ms
64 bytes from 192.168.20.1: seq=2 ttl=64 time=1.211 ms
64 bytes from 192.168.20.1: seq=3 ttl=64 time=0.719 ms

--- 192.168.20.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.666/0.816/1.211 ms
/ # ping -c 4 192.168.10.253
PING 192.168.10.253 (192.168.10.253): 56 data bytes
64 bytes from 192.168.10.253: seq=0 ttl=63 time=1.573 ms
64 bytes from 192.168.10.253: seq=1 ttl=63 time=2.017 ms
64 bytes from 192.168.10.253: seq=2 ttl=63 time=2.392 ms
64 bytes from 192.168.10.253: seq=3 ttl=63 time=1.384 ms

--- 192.168.10.253 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 1.384/1.841/2.392 ms
/ #
```

Рисунок 16 — Проверка PC2

```

ivan@ivan-asus: ~/pmut/lab1
[admin@R01.TEST] > /ip address print
Flags: X - disabled, I - invalid, D - dynamic
# ADDRESS NETWORK INTERFACE
0 172.31.255.30/30 172.31.255.28 ether1
1 192.168.10.1/24 192.168.10.0 vlan10
2 192.168.20.1/24 192.168.20.0 vlan20
[admin@R01.TEST] > /ip dhcp-server lease print
Flags: X - disabled, R - radius, D - dynamic, B - blocked
# ADDRESS MAC-ADDRESS HOST.. SERVER RAT.. STATUS LAST-SEEN
0 D 192.168.10.253 AA:C1:AB:F3:0D:EF dhcp10 bound 1m17s
1 D 192.168.20.253 AA:C1:AB:30:0C:02 dhcp20 bound 1m2s
[admin@R01.TEST] > /ping 192.168.10.253 count=4
SEQ HOST SIZE TTL TIME STATUS
0 192.168.10.253 56 64 0ms
1 192.168.10.253 56 64 1ms
2 192.168.10.253 56 64 1ms
3 192.168.10.253 56 64 0ms
sent=4 received=4 packet-loss=0% min-rtt=0ms avg-rtt=0ms max-rtt=1ms
[admin@R01.TEST] > /ping 192.168.20.253 count=4
SEQ HOST SIZE TTL TIME STATUS
0 192.168.20.253 56 64 0ms
1 192.168.20.253 56 64 0ms
2 192.168.20.253 56 64 0ms
3 192.168.20.253 56 64 0ms
sent=4 received=4 packet-loss=0% min-rtt=0ms avg-rtt=0ms max-rtt=0ms
[admin@R01.TEST] > |

```

Рисунок 17 — Проверка маршрутизатора

```

ivan@ivan-asus: ~/pmut/lab1
[admin@SW01.L3.01.TEST] > /interface ethernet print
Flags: X - disabled, R - running, S - slave
# NAME MTU MAC-ADDRESS ARP
0 R ether1 1500 0C:00:5F:37:5E:00 enabled
1 RS ether2 1500 AA:C1:AB:C2:E8:C6 enabled
2 RS ether3 1500 AA:C1:AB:D7:0C:FD enabled
3 RS ether4 1500 AA:C1:AB:4C:73:11 enabled
[admin@SW01.L3.01.TEST] > /interface bridge host print
Flags: X - disabled, I - invalid, D - dynamic, L - local, E - external
# MAC-ADDRESS VID ON-INTERFACE BRIDGE AGE
0 DL AA:C1:AB:4C:73:11 ether4 bridge1
1 DL AA:C1:AB:C2:E8:C6 ether2 bridge1
2 DL AA:C1:AB:D7:0C:FD ether3 bridge1
3 D AA:C1:AB:19:E1:4D 1 ether4 bridge1 0s
4 D AA:C1:AB:40:4C:71 1 ether3 bridge1 19s
5 DL AA:C1:AB:4C:73:11 1 ether4 bridge1
6 DL AA:C1:AB:C2:E8:C6 1 ether2 bridge1
7 DL AA:C1:AB:D7:0C:FD 1 ether3 bridge1
8 D AA:C1:AB:F4:4C:1F 1 ether2 bridge1 19s
9 DL AA:C1:AB:C2:E8:C6 10 ether2 bridge1
10 DL AA:C1:AB:D7:0C:FD 10 ether3 bridge1
11 D AA:C1:AB:F3:0D:EF 10 ether3 bridge1 1m43s
12 D AA:C1:AB:F4:4C:1F 10 ether2 bridge1 19s
13 D AA:C1:AB:30:0C:02 20 ether4 bridge1 1m37s
14 DL AA:C1:AB:4C:73:11 20 ether4 bridge1
15 DL AA:C1:AB:C2:E8:C6 20 ether2 bridge1
16 D AA:C1:AB:F4:4C:1F 20 ether2 bridge1 19s
[admin@SW01.L3.01.TEST] > /interface bridge vlan print
Flags: X - disabled, D - dynamic
# BRIDGE VLAN-IDS CURRENT-TAGGED CURRENT-UNTAGGED
0 D bridge1 1 bridge1
ether2
ether3
ether4
1 bridge1 10 ether2
ether3
2 bridge1 20 ether2
ether4
[admin@SW01.L3.01.TEST] > |

```

Рисунок 18 — Проверка центрального коммутатора

ЗАКЛЮЧЕНИЕ

В ходе работы был изучен инструмент ContainerLab и методы работы с ним, возобновлены знания о работе VLAN, IP адресации, DHCP, был установлен docker, make, containerlab, собран контейнер с RouterOS, собрана трёхуровневая сеть связи, настроены маршрутизатор и коммутаторы.

В процессе возникали трудности с установкой на компьютере из-за вложенной виртуализации и ошибки из-за невнимательности.